

MECHATRONICS AND IOT

ASSIGNMENT REPORT-2

TITLE:

Design and develop an MQTT-Based Remote HVAC Monitoring System using ThingSpeak Cloud. The system remotely monitors HVAC parameters and stores sensor data on cloud platforms for analysis and control

DONE BY ,

HARI KRISHNA JAGADEESH-24MER011

HARISH DHARMALINGAM – 24MER014

RANJITH KUMAR P – 24MER048

RISHI KESAV A – 24MER049

VENKATRAMANAN B – 24MER061

Project Overview:

The Remote HVAC Monitoring System is an IoT-based system designed to continuously monitor important environmental parameters such as temperature and humidity in an HVAC environment. The system uses a DHT11 sensor to measure the temperature and humidity, while an ESP8266 NodeMCU acts as the main controller for collecting and processing the sensor data.

The measured temperature and humidity values are displayed locally on a 0.96-inch OLED display, allowing users to monitor the HVAC conditions in real time. The system can identify abnormal temperature conditions and provide the corresponding status on the display.

In the complete system, the ESP8266 can use its built-in Wi-Fi connectivity and MQTT protocol to transmit the measured HVAC parameters to a cloud platform such as ThingSpeak. The cloud platform can store, visualize and analyze the collected data, enabling remote monitoring of HVAC performance.

The proposed system provides a low-cost, compact and scalable solution for HVAC monitoring and can be extended to include automatic fan control, alarms, data logging and remote HVAC control.

Problem Statement:

Heating, Ventilation and Air Conditioning (HVAC) systems play an important role in maintaining comfortable and healthy indoor environments. However, continuously monitoring parameters such as temperature and humidity is often difficult, especially when the HVAC system is located in industrial, commercial or large indoor spaces.

Conventional monitoring methods may require manual observation and do not provide continuous remote access to environmental data. This can result in delayed detection of abnormal temperature or humidity conditions, leading to reduced comfort, inefficient HVAC operation and unnecessary energy consumption.

Therefore, there is a need for a low-cost IoT-based HVAC monitoring system that can continuously measure temperature and humidity, display the parameters locally, transmit the data remotely and store the collected information on a cloud platform for real-time monitoring, analysis and future HVAC control.

Proposed Solution:

The proposed system is an IoT-based remote HVAC monitoring system that uses an ESP8266 NodeMCU as the main controller and a DHT11 sensor to measure temperature and humidity continuously. The sensor data is collected by the ESP8266 and displayed locally on a 0.96-inch OLED display, allowing users to observe the current HVAC conditions in real time.

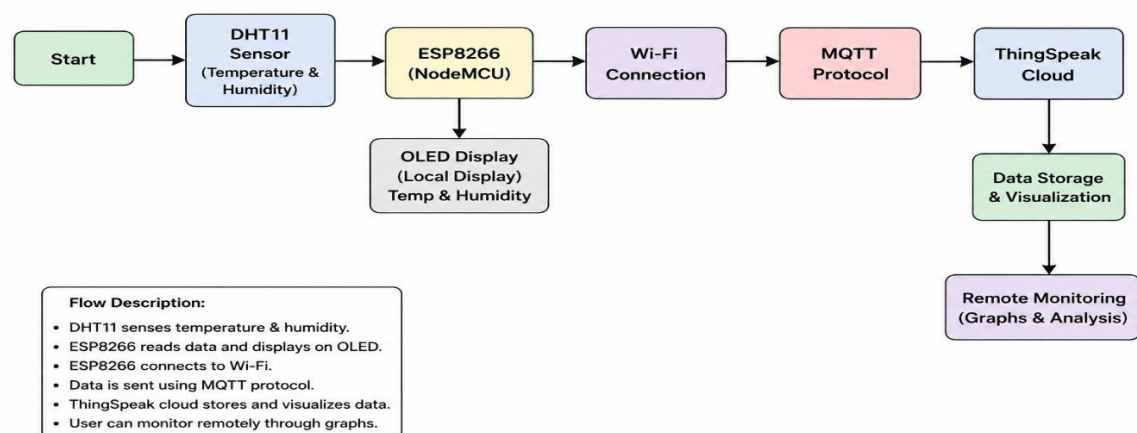
The ESP8266 uses its built-in Wi-Fi connectivity to transmit the collected sensor data using the MQTT communication protocol. An MQTT broker manages the communication between the ESP8266 and the cloud platform. ThingSpeak Cloud is used to receive, store and visualize the HVAC parameters through graphs and dashboards.

The system can also compare the measured temperature with predefined limits to identify abnormal conditions. In future development, the ESP8266 can control a fan or HVAC actuator automatically based on the monitored conditions.

The proposed solution provides a low-cost, real-time and scalable HVAC monitoring system that supports both local and remote monitoring, data storage and analysis.

System Architecture:

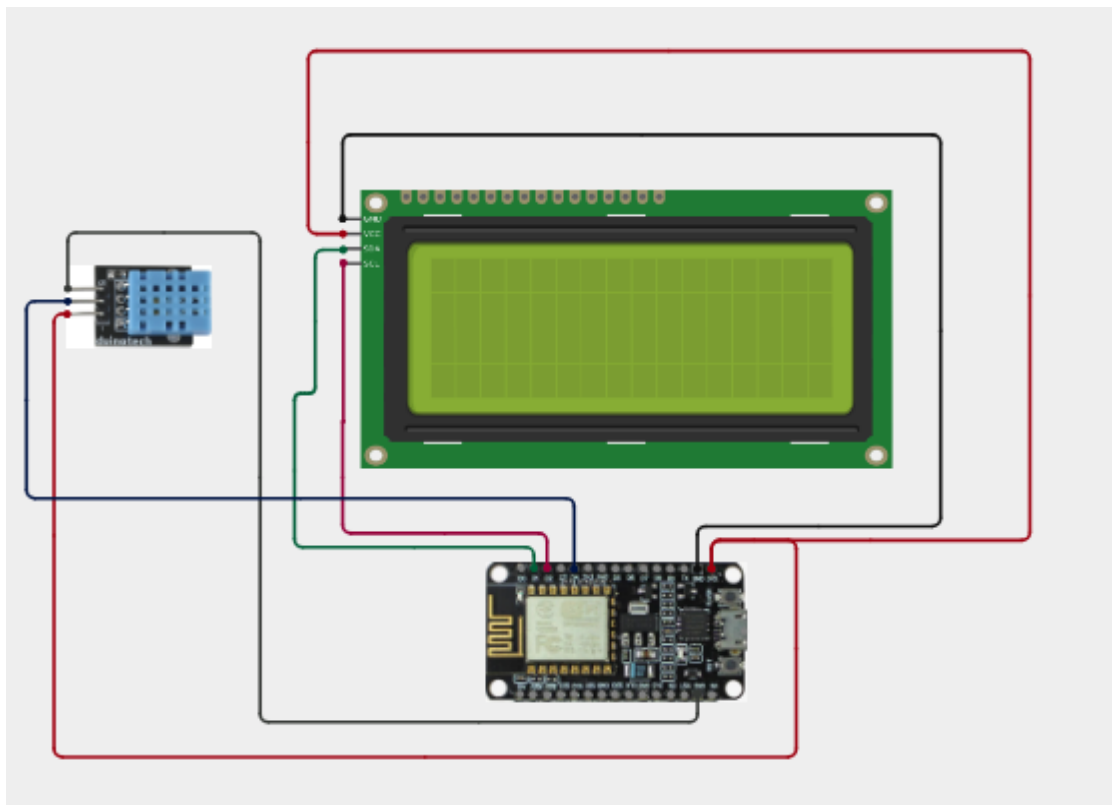
SYSTEM ARCHITECTURE FLOWCHART



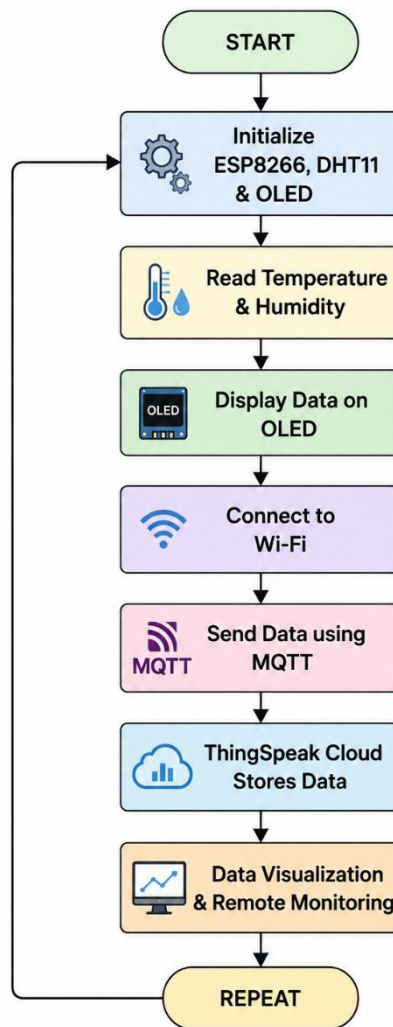
Circuit Table :

Component	Pin/Terminal	ESP8266 Connection	Purpose
DHT 11 Sensor	VCC	3.3V	Power Supply
LCD Display	VCC	3.3V	Safe indication
ESP8266	VIN/USB	5V USB	Hazardous indication

Circuit Design:



Algorithm:



Coding:

```
#include <Wire.h>
```

```
#include <LiquidCrystal_I2C.h>
```

```
#include <DHT.h>
```

```
#include <ESP8266WiFi.h>
```

```
#include <ThingSpeak.h>
```

```
// =====
```

```
// WIFI SETTINGS
```

```
// =====  
  
const char* ssid = "Xiaomi 11i";  
const char* password = "12345678";  
WiFiClient client;  
  
// =====  
  
// THINGSPEAK SETTINGS  
  
// =====  
  
unsigned long channelID = 3456112;  
const char* writeAPIKey = "T4K4D4CK5S4ISADC";  
  
// =====  
  
// DHT11 SETTINGS  
  
// =====  
  
#define DHT_PIN 02  
  
#define DHT_TYPE DHT11  
  
DHT dht(DHT_PIN, DHT_TYPE);  
  
// =====  
  
// LCD SETTINGS  
  
// =====  
  
LiquidCrystal_I2C lcd(0x27, 16, 2);  
  
// =====  
  
// SETUP  
  
// =====  
  
void setup()  
{  
    Serial.begin(115200);  
  
    // Start DHT11  
  
    dht.begin();  
  
    // Start I2C
```

```
// SDA = GPIO4 (D2)
// SCL = GPIO5 (D1)
Wire.begin(04, 05);
// Start LCD
lcd.init();
lcd.backlight();
lcd.clear();
lcd.setCursor(2, 0);
lcd.print("HVAC MONITOR");
lcd.setCursor(1, 1);
lcd.print("Connecting...");
// =====
// CONNECT TO WIFI
// =====
WiFi.begin(ssid, password);
Serial.println();
Serial.print("Connecting to WiFi");
while (WiFi.status() != WL_CONNECTED)
{
    delay(500);
    Serial.print(".");
}
Serial.println();
Serial.println("WiFi Connected!");
Serial.print("IP Address: ");
Serial.println(WiFi.localIP());
// =====
// CONNECT TO THINGSPEAK
// =====
```

```

ThingSpeak.begin(client);

Serial.println("ThingSpeak Connected!");

lcd.clear();

lcd.setCursor(0, 0);

lcd.print("WiFi Connected");

lcd.setCursor(0, 1);

lcd.print("Cloud Ready");

delay(2000);

}

// =====

// LOOP

// =====

void loop()

{

// =====

// READ DHT11

// =====

float temperature = dht.readTemperature();

float humidity = dht.readHumidity();

// =====

// CHECK SENSOR

// =====

if (isnan(temperature) || isnan(humidity))

{

Serial.println("DHT11 reading failed!");

lcd.clear();

lcd.setCursor(2, 0);

lcd.print("SENSOR ERROR");

```

```
    lcd.setCursor(0, 1);  
    lcd.print("Check DHT11");  
    delay(2000);  
    return;  
}  
// =====  
// HVAC STATUS  
// =====  
  
int statusValue;  
  
if (temperature > 30)  
{  
    statusValue = 2;  
}  
  
else if (temperature < 20)  
{  
    statusValue = 0;  
}  
  
else  
{  
    statusValue = 1;  
}  
  
// =====  
  
// SERIAL MONITOR  
// =====  
  
Serial.print("Temperature: ");  
Serial.print(temperature);  
Serial.println(" C");  
Serial.print("Humidity: ");  
Serial.print(humidity);
```

```
Serial.println(" %");
Serial.print("HVAC Status: ");
if (statusValue == 2)
{
    Serial.println("HIGH");
}
else if (statusValue == 0)
{
    Serial.println("LOW");
}
else
{
    Serial.println("NORMAL");
}

// =====

// LCD DISPLAY

// =====

lcd.clear();

// Line 1

lcd.setCursor(0, 0);
lcd.print("T:");
lcd.print(temperature, 1);
lcd.print((char)223);
lcd.print("C");
lcd.setCursor(9, 0);
lcd.print("H:");
lcd.print(humidity, 1);
lcd.print("%");

// Line 2
```

```
lcd.setCursor(0, 1);
if (temperature > 30)
{
    lcd.print("Status: HIGH");
}
else if (temperature < 20)
{
    lcd.print("Status: LOW");
}
else
{
    lcd.print("Status: NORMAL");
}

// =====
// SEND TO THINGSPEAK
// =====

ThingSpeak.setField(1, temperature);
ThingSpeak.setField(2, humidity);
ThingSpeak.setField(3, statusValue);
int response = ThingSpeak.writeFields(
    channelID,
    writeAPIKey
);
if (response == 200)
{
    Serial.println("Cloud upload successful!");
}
else
{

```

```

    Serial.print("Cloud upload failed. Error: ");
    Serial.println(response);
}
Serial.println("-----");
// =====
// WAIT
// =====
delay(20000);
}

```

System Working:

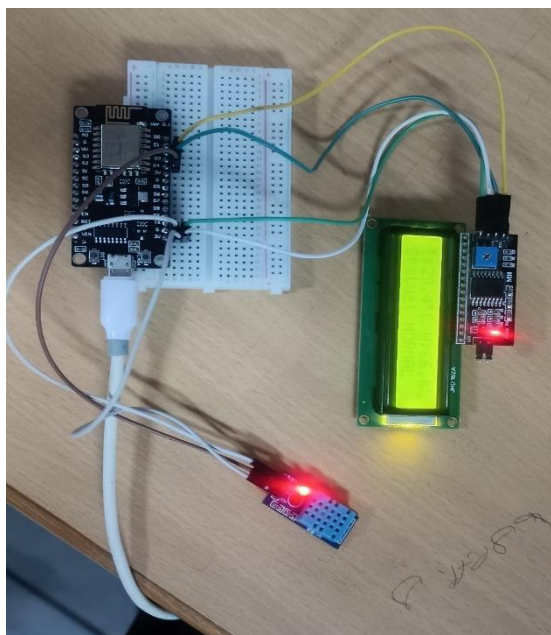
- The system uses a DHT11 sensor to continuously measure the temperature and humidity of the HVAC environment. The sensor readings are collected and processed by the ESP8266 NodeMCU.
- The measured temperature and humidity values are displayed locally on the OLED display, allowing the user to monitor the HVAC conditions in real time. The ESP8266 then connects to a Wi-Fi network and prepares the sensor data for remote transmission.
- Using the MQTT protocol, the ESP8266 sends the collected HVAC data to the cloud platform. ThingSpeak Cloud receives and stores the data, which can then be represented using graphs and charts for remote monitoring and analysis.
- The system continuously repeats the sensing, processing, display and cloud-monitoring process, providing real-time HVAC parameter monitoring. The system can also be extended in the future to automatically control fans, air-conditioning units or other HVAC equipment based on predefined temperature and humidity limits.

Bill Of Material:

S. no	Component	Specification	Quantity	Cost
1	ESP8266	Node MCU ESP8266	1	Rs. 450
2	DHT11 Sensor	Temperature & Humidity Sensor	1	Rs. 100
3	OLED Display	0.96" I ² C, SSD1306, 128×64	1	Rs. 150
5	Breadboard	Standard 830-point	1	Rs. 100
6	Jumper Wire	Male-Male/Male-Female	1 set	Rs. 50
			Total	Rs. 850

Prototype Implementation:

- The prototype was implemented using an ESP8266 NodeMCU, DHT11 temperature and humidity sensor, and 0.96-inch OLED display. The DHT11 sensor was connected to the ESP8266 to continuously measure the temperature and humidity of the surrounding HVAC environment.
- The ESP8266 processes the sensor readings and displays the measured values on the OLED screen in real time. The built-in Wi-Fi capability of the ESP8266 is used to establish a wireless connection for remote monitoring.



Results and Performance:

- The developed MQTT-Based Remote HVAC Monitoring System successfully monitored the temperature and humidity of the surrounding environment using the DHT11 sensor and ESP8266 NodeMCU. The measured parameters were displayed on the OLED display, providing real-time local monitoring.
- The system demonstrated reliable sensor data acquisition and continuous updating of the displayed values. The ESP8266 also provided Wi-Fi connectivity for transmitting the collected data through the MQTT protocol to the ThingSpeak Cloud. The cloud platform enabled data storage, graphical visualization and remote monitoring of HVAC conditions.
- The prototype provides a low-cost, reliable and scalable solution for HVAC monitoring. The system can be further improved by using higher-accuracy sensors, adding automatic HVAC control, implementing alerts for abnormal conditions and incorporating additional parameters such as air quality and energy consumption.